

Documentation technique SUPCHAT



Développeur :

- Martorell Rémi
- DUTREUIL Hugo

Table des matières

Informations Générales	3
Introduction	3
Architecture Générale	3
Les services Backend	4
Déploiement	4
Serveur HTTP (Ngnix)	5
Communication avec l'API	6
Authentications et Gestions des sessions	7
Connexion classique	8
Connexion avec OAuth2	9
Messagerie instantanée et gestions des utilisateurs	11
Envoie de fichier	12
Workspace & Canaux	13
Création / Gestion des Workspace	13
Channels public et privé	14
Notification en temps réel	16
Api externe	17
Barre de recherche avancé	18
Interface utilisateur	20
Développement mobile	20

Informations Générales

Introduction

Dans un contexte numérique très fort accélérer par le télétravail depuis le COVID-19. L'augmentations exponentiel du distanciel ne fait qu'augmenter ces dernières années. Les entreprises se doivent de s'adopter de solution innovante, efficaces en manière de communications et de respect de sécurité des données. C'est dans cette perspective que le projet SUPCHAT intervient. Il répond précisément aux besoins exprimés dans le cahier des charges. SUPCHAT n'est pas qu'un simple outil de messagerie instantané, c'est un véritable écosystème avec divers fonctionnalités indispensable pour le travail collaboratif. Il dispose de messagerie privée, des canaux pour des discussions de groupes avec plusieurs employés avec des discussions clairs et précises. Pour répondre pleinement à ces enjeux variés, notre équipe a conçu une solution technique robuste et évolutive facilement, reposant sur des technologies moderne. Ainsi, SUPCHAT est architecturé autour d'une API REST performante, d'une base de données non relationnelle flexible, et propose 2 interfaces distinctes une application web réactive, accessible depuis tout navigateur, et une application mobile compatible avec Android. Cette combinaison technologique permet de couvrir tous les cas d'utilisation possibles tout en garantissant une expérience utilisateur optimale et cohérente, indépendamment du dispositif utilisé.

Architecture Générale

- Backend : API REST en Node.js avec Express.js
- Base de données : MongoDB
- Frontend : React
- Application Mobile : React Native
- Communication en temps réel : Socket.IO (les WebSockets)
- Authentifications : OAuth2(Google/Facebook) et authentification classique via JWT
- Sécurité : Bcrypt, CORS, HTTPS
- Déploiement : Docker ou via le VPS grâce à notre hébergeur IONOS

Les services Backend

Le service client-web joue un rôle très important en fournissant notamment l'interface utilisateur. Il est déployé dans un conteneur Docker, utilisant une image Nginx comme serveur HTTP statique pour héberger l'application après une phase de build.

Déploiement

Pour répondre le plus facilement possible aux exigences de déploiement stable, sécurisé et évolutifs, le projet SUPCHAT est une architecture conteneurisée utilisant Docker géré avec le docker compose. Cela permet d'assurer une installation

Rapide et fiable sur n'importe quel hébergeur. Cela permet ensuite à notre VPS de déployer notre VPS. Le déploiement final est disponible en ligne sur <https://supchat.info>.

L'architecture est divisée en trois grands services. Chacun fonctionne dans un conteneur Docker indépendant. Cette approche modulaire favorise la flexibilité, et la sécurité entre les différents composants applicatifs :

- Backend : API REST qui gère toutes les requêtes provenant des clientsweb et mobile, les authentifications, les espaces de travaux, les canaux, les messages, les notifications et pleins d'autres.
- Frontend : interface web développé avec React, qui réagit directement avec le serveur API.
- Bases de données : MongoDB est configuré pour stocker durablement toutes les données utilisateurs, messages permissions et toute autres encore.

Serveur HTTP (Nginx)

Le choix de Nginx c'est favorisé avec ces capacités à se servir du contenu statique et de manière la plus fiable possible en sécurité. Nginx est configuré pour servir directement les fichiers statiques produit par le build React pour améliorer au maximum les performances.

```
2  upstream supchat-backend {
3      server supchat-backend:3000;
4  }
5
6  server {
7      listen      80;
8      server_name 192.168.5.180;
9
10     root    /usr/share/nginx/html;
11     index   index.html;
12
13     # WebSocket / Socket.IO
14     location /socket.io/ {
15         proxy_pass          http://supchat-backend;
16         proxy_http_version  1.1;
17         proxy_set_header    Upgrade $http_upgrade;
18         proxy_set_header    Connection "Upgrade";
19         proxy_set_header    Host $host;
20         # évite que nginx ferme la connexion trop tôt
21         proxy_read_timeout  86400s;
22     }
23
24     # API REST / Auth
25     location ~ ^/(api|auth)/ {
26         proxy_pass          http://supchat-backend;
27         proxy_set_header    Host $host;
28         proxy_set_header    X-Real-IP $remote_addr;
29         proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
30     }
```

Voici le docker file du processus du build :

```
2 >> FROM node:20 AS build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm install
6 COPY . .
7 RUN npm run build
8
9
10 # Étape 2 : servir via NGINX avec configuration personnalisée
11 FROM nginx:alpine
12 # Copier le build de React
13 COPY --from=build /app/build /usr/share/nginx/html
14 # Copier votre configuration Nginx personnalisée
15 COPY nginx.conf /etc/nginx/conf.d/default.conf
16 EXPOSE 80
17 CMD ["nginx", "-g", "daemon off;"]
```

Communication avec l'API

Pour gérer facilement les communications HTTP avec l'API, l'application utilise la librairie Axios. Celle-ci a été choisie pour simplicité d'usage, sa gestion robuste des erreurs et ces capacités avancées sur la gestion des requêtes JWT d'authentications.

Avantage :

Gestion des requêtes :

- Injection automatique des token JWT pour sécuriser toutes les requêtes envoyées à l'API.
- Traitement centralisé des erreurs pour fournir une expérience utilisateur claire.
- Gestions efficaces des requêtes RESTful (GET, POST, PUT & DELETE).

```
1  axios.interceptors.request.use(
2    (config) => {
3      const token = localStorage.getItem('jwtToken');
4      if (token) {
5        config.headers.Authorization = `Bearer ${token}`;
6      }
7      return config;
8    },
9    (error) => Promise.reject(error)
10 );
```

Authentifications et Gestions des sessions

Notre site web dispose de 2 types de connexion, la première est la méthode classique via adresse mail. La 2^e méthode est via OAuth lié avec google et Facebook qui gère une connexion rapide, simple et sécurisé.

Inscription classique

```
router.post('/register', { handlers: async (req: Request<P, ResBody, ReqBody, ReqQuery, LocalsObj>, res: Response<ResBody, LocalsObj>) => {
  try {
    const { name, email, password, acceptedTerms } = req.body;
    if (!acceptedTerms) {
      return res.status(400).json({ msg: 'Vous devez accepter les Conditions d'utilisation & RGPD.' });
    }
    let user: Query<...> & ObtainSchemaGeneric<module:mon...> = await User.findOne({ filter: { email } });
    if (user) {
      return res.status(400).json({ msg: 'Un compte existe déjà avec cette adresse email.' });
    }
    user = new User({ doc: { name, email, password, isVerified: false } });
    const verificationToken: string = crypto.randomBytes(20).toString('hex');
    user.verificationToken = verificationToken;
    user.emailVerificationExpires = Date.now() + 24 * 3600 * 1000; // 24h
    await user.save();

    const verifyURL: string = `${process.env.BACK_URL}/api/auth/verify-email?token=${verificationToken}`;
    const mailOptions: { ... } = {
      from: 'Support SupChat <contact@supchat.info>',
      to: user.email,
      subject: 'Confirmez votre compte SupChat',
      html: `
        <h1>Bienvenue sur SupChat</h1>
        <p>Merci de vous être inscrit. Cliquez sur le lien ci-dessous pour confirmer votre compte :</p>
        <a href="${verifyURL}">Activer mon compte</a>
        <p>Ce lien expire dans 24 heures.</p>
      `;
    };
    try {
      await transporter.sendMail(mailOptions);
    }
  }
});
```

La méthode dite classique, fonctionne avec une adresse mail valide avec une vérification dans le champ d'un « @ » et d'un « . », pour éviter la saturation de fausse adresse mail. Les autres champs à remplir son pseudo unique et un mot de passe avec des règles 8 caractères au minimum un chiffre et un caractère spécial. Les mots de passes ne sont pas stockés directement dans la base de données, pour la sécurité tous les mots de passes sont hachés avec bcrypt. Lorsque la création a été réussie le compte doit être vérifié grâce à un protocole SMTP, qui enverra automatiquement un mail à l'utilisateur pour confirmer son compte avec un jeton d'une limite de 24h. Cette méthode consiste à éviter toute usurpation de compte ou d'intervention malveillante que l'utilisateur sera inscrit uniquement si le compte a été vérifié.

```

const transporter = nodemailer.createTransport({ transporter: {
  host: process.env.EMAIL_HOST, // ex. smtp.ionos.fr
  port: parseInt(process.env.EMAIL_PORT, radix: 10), // 587
  secure: process.env.EMAIL_SECURE === 'true', // false pour STARTTLS
  auth: {
    user: process.env.EMAIL_USER, // contact@supchat.info
    pass: process.env.EMAIL_PASS, // mot de passe
  },
  // Optionnel : TLS
  tls: {
    rejectUnauthorized: false,
  },
});

module.exports = transporter;

```

Connection classique

Lorsque l'utilisateur c'est déjà inscrit, il pourra alors se connecter à l'application SUPCHAT. Il devra alors saisir son mail ainsi que son mot de passe sur le formulaire de connexion. L'API de connexion va alors vérifier que l'utilisateur existe bien dans la base de données, puis il comparera le mot de passe avec le hashage bcrypt stocké, il refusera l'accès avec une erreur 401 en cas d'échec et enverra aussi un message d'erreur si le compte n'a pas été vérifié. Il générera un jeton JWT signé si l'utilisateur a rentré un mot de passe valide avec un temps d'expiration de 8h stocké localement dans la machine pour éviter de se reconnecter systématiquement.


```

router.post( path: '/Login', handlers: async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<ResBody, LocalsObj>)
  try {
    const { email, password } = req.body;
    const user : Query<...> & ObtainSchemaGeneric<module:mon... = await User.findOne( filter: { email });
    if (!user) return res.status( code: 400).json( body: { msg: 'Identifiants invalides' });
    if (!user.isVerified) {
      return res.status( code: 403).json( body: { msg: 'Votre compte n'est pas activé. Vérifiez vos emails.' });
    }
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status( code: 400).json( body: { msg: 'Identifiants invalides' });

    const payload : {user: {...}} = { user: { id: user._id } };
    const token = jwt.sign(payload, process.env.JWT_SECRET, options: { expiresIn: '8h' });
    return res.json( body: {
      token,
      user: {
        _id: user._id,
        name: user.name,
        email: user.email,
        theme: user.theme
      }
    });
  } catch (err) {
    console.error('✖ Erreur login :', err);
    return res.status( code: 500).json( body: { msg: 'Erreur serveur' });
  }
});

```

Connexion avec OAuth2

Il est possible de se connecter directement via des OAuth2 avec Facebook et Google. Cette méthode facilite la connexion pour les utilisateurs en gagnant du temps. Lorsque l'utilisateur veut se connecter via Google ou Facebook, il sera redirigé http 302 vers le Endpoint de notre server qui gèrera la stratégie de connections grâce au Passeport. L'utilisateur lorsqu'il c'est connecter via Google ou Facebook, il sera automatiquement redirigé sur son compte en recevant un token de connexion, avec toutes ces informations renseigner sur OAuth2 Email et pseudo. En cas d'erreur le passeport stop le processus et renvoie une erreur 401. La sécurisation des secret, les clé API de Facebook et de Google ne sont pas stocké directement dan le fichier mais dans un dossier dédié le .env.

```

const passport : Authenticator | {...} = require('passport');
const FacebookStrategy : function(Object, Function): vo... | {...} = require('passport-facebook').Strategy;
const GoogleStrategy : function(Object, Function): vo... | {...} = require('passport-google-oauth20').Strategy;
const User : ... = require('../models/User');

// --- FACEBOOK STRATEGY ---
passport.use(new FacebookStrategy({ options: {
  clientId: process.env.FACEBOOK_APP_ID,
  clientSecret: process.env.FACEBOOK_APP_SECRET,
  callbackURL: `${process.env.BACK_URL}/api/auth/facebook/callback`,
  profileFields: ['id', 'emails', 'name', 'displayName', 'photos']
},
  verify: async (accessToken, refreshToken, profile, done) : Promise<void> => {
    try {
      let user : Query<...> & ObtainSchemaGeneric<module:mon... = await User.findOne({ filter: { facebookId: profile.id }});
      if (!user) {
        user = new User({ doc: {
          facebookId: profile.id,
          name: profile.displayName ||
            `${profile.name.givenName} ${profile.name.familyName}`,
          email: profile.emails?.[0].value,
          isVerified: true,
          password: undefined
        }});
        await user.save();
      }
      done(null, user);
    } catch (err) {
      done(err);
    }
  }
});

```

Messagerie instantanée et gestions des utilisateurs

La messagerie instantanée utilise la bibliothèque Socket.IO. Elle permettant des interactions entre le serveur et les clients connectés. Cette technologie repose sur le protocole WebSocket, complété par des mécanismes de fallback assurant une robustesse optimale. C'est cette partie de code dans `supchat-backend/index.js` :

```
151 // Gestion des connexions utilisateurs
152 socket.on('join', userId => {
153   if (!userId) return;
154   userSocketMap[userId] = socket.id;
155   socket.emit('joined', { message: `Connecté via ${socket.id}` });
156   socket.broadcast.emit( ev: 'user-connected', userId);
157   socket.emit('active-users', Object.keys(userSocketMap));
158 });
159
160 socket.on('joinChannel', channelId => {
161   socket.join(channelId);
162   console.log(`✳️ ${socket.id} rejoint channel ${channelId}`);
163 });
164
165 socket.on('sendMessage', ({ channelId, message }) : void => {
166   io.to(channelId).emit('receiveMessage', { message, channelId });
167 });
168
169 socket.on('sendDirectMessage', ({ toUserId, message }) : void => {
170   const sid = userSocketMap[toUserId];
171   if (sid) io.to(sid).emit('receiveDirectMessage', { message });
172 });
173
174 socket.on('disconnect', () : void => {
175   console.log(`⚠️ Déco : ${socket.id}`);
176   for (const [u : string, sid] of Object.entries(userSocketMap)) {
177     if (sid === socket.id) {
178       delete userSocketMap[u];
179       socket.broadcast.emit( ev: 'user-disconnected', u);
180       break;
181     }
182   }
183 }
```

Envoie de fichier

La gestion de l'envoi de fichier sont effectuée avec une intégration de multer sur coté serveur, qui va gérer les uploads et le stockage des fichiers sur le serveur de manière sécurisé.

```
102 // 1 Création et sauvegarde du DM
103 const newMessage : HydratedDocument<...> | HydratedDocument<...> = new DirectMessage({
104   sender: req.user.id,
105   receiver: receiverId,
106   content,
107   fileUrl: req.file ? `/uploads/${req.file.filename}` : null,
108 });
109 const message : any = await newMessage.save();
110
111 // 2 Emission du DM vers l'expéditeur et le destinataire
112 const io = req.app.get('socketio');
113 const userSocketMap = req.app.get('userSocketMap');
114 const senderSock = userSocketMap[req.user.id];
115 const receiverSock = userSocketMap[receiverId];
116
117 const payload : {...} = {
118   _id: message._id,
119   sender: req.user.id,
120   receiver: receiverId,
121   content: message.content,
122   fileUrl: message.fileUrl,
123   createdAt: message.createdAt
124 };
125 if (senderSock) io.to(senderSock).emit('new-private-message', payload);
126 if (receiverSock) {
127   io.to(receiverSock).emit('new-private-message', payload);
128   console.log(`Message privé émis à ${receiverId} via socket ${receiverSock}`);
129 }
130
```

Une limite de 5 Mo a été imposé pour ne pas saturer les serveurs dans stockages des fichiers et rendre l'application plus fluide. La limite de stockage peut tout à fait être augmenté selon vos besoins.

```
supchat-backend/index.js  workspaces.js  users.js  DirectMessage.js  directMessages.js  upload.js  channels.js  messages.js
1 //middleware/uploads.js
2 const multer : function(any): (Multer | undef...) | {...} = require("multer");
3
4 const storage : MemoryStorage = multer.memoryStorage(); // stocke le fichier en RAM, on le transfère manuellement après
5 const upload : Multer = multer({ options: {
6   storage,
7   limits: { fileSize: 5 * 1024 * 1024 }, // 5 Mo max
8 }});
9
10 module.exports = upload;
11
```

Workspace & Canaux

Création / Gestion des Workspace

La création de Workspace, ce fait grâce à des requêtes POST avec l'API Workspace qui va créer un document Mongoose en inscrivant l'utilisateur qui a créé le workspace en owner qui a le rôle du propriétaire et renverra un code http 201 pour confirmer la bonne création du Workspace. L'API retourne tous les workspaces auxquels l'utilisateur est affilié, chaque objet inclut ses canaux, la liste des membres et leur rôle, et les métadonnées (date de création, description éventuelle).

```
router.post( path: '/', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Respon

    const { name } = req.body;

    try {
        const newWorkspace : HydratedDocument<InferSchemaType<...> = new Workspace( doc: {
            name,
            owner: req.user.id,
            members: [
                {
                    user: req.user.id,
                    role: 'owner'
                }
            ],
        });

        const workspace :... = await newWorkspace.save();
        res.json(workspace);
    } catch (err) {
        console.error(err.message);
        res.status( code: 500).send( body: 'Server Error');
    }
});
```

Les utilisateurs peuvent disposer de 4 rôles sur les workspaces, le premier rôle est le Owner qui est attribué par défaut à l'utilisateur qui a créé le workspace. Le rôle admin permet de gérer els server créer supprimer et gérer les utilisateurs, alors que le modérateur peut juste gérer les utilisateurs et le membre ne dispose que d'aucun rôle en particulier.

```
// Petite fonction de hiérarchie
function roleRank(role) : number { Show usages  Rémi Martorell
  switch (role) {
    case 'owner': return 4;
    case 'admin': return 3;
    case 'moderator': return 2;
    case 'member': return 1;
    default: return 0;
  }
}
```

Par défaut la totalité des utilisateurs du workspace ajouté sont membre.

```
router.get(path: '/', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> ) : Promise<void> => {  Rémi Martorell
  try {
    const workspaces : Query<...> & ObtainSchemaGeneric<module:mon... = await Workspace.find( filter: { "members.user": req.user.id });
    res.json(workspaces);
  } catch (err) {
    console.error(err.message);
    res.status( code: 500 ).send( body: 'Server Error' );
  }
});
```

Channels public et privé

Pour la création de workspace, le client envoie un POST vers `/api/channels/:workspaceId/channels` en passant, dans le corps, le nom du canal, son type (public ou private), pour un canal privé, la liste des membres à inviter sera proposer à l'utilisateur.

```

35 router.post( path: './workspaceId/channels', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> )
36   const { name, type, members } = req.body;
37   try {
38     const workspace : Query<...> & ObtainSchemaGeneric<module:mon...> = await Workspace.findById(req.params.workspaceId);
39     if (!workspace) return res.status( code: 404 ).json( body: { msg: 'Workspace not found' } );
40
41     const isMember = workspace.members.some(m => m.user.toString() === req.user.id);
42     if (!isMember) return res.status( code: 403 ).json( body: { msg: 'Access denied' } );
43
44     const newChannel : HydratedDocument<InferSchemaType<...>> = new Channel( doc: {
45       name,
46       type: type || 'public',
47       workspace: req.params.workspaceId,
48       members: (type === 'private') ? [req.user.id, ...(members || [])] : []
49     });
50
51     await newChannel.save();
52
53     const io = req.app.get('socketio');
54     const userSocketMap = req.app.get('userSocketMap');
55     workspace.members.forEach(member => {
56       const socketId = userSocketMap[member.user.toString()];
57       if (socketId) io.to(socketId).emit('channel-added', newChannel);
58     });

```

Une vérification est assurée pour que seul le owner et les admins peuvent créer un channels. Si un membre ou un modérateur créer un channels-il un message «Acces denied». Il est possible de crée des channels privé où juste certain membre de votre workspace aurons accès. Si l'utilisateur essaye de se connecter à un channels privé via l'id sur le navigateur, il ne pourra pas se connecter du a la vérifications de l'API :

```

Récupérer les canaux d'un workspace
~ ter.get( path: './workspaceId/channels', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> ) : Promise<...> => {
  ~ try {
    const workspace : Query<...> & ObtainSchemaGeneric<module:mon...> = await Workspace.findById(req.params.workspaceId);
    if (!workspace) return res.status( code: 404 ).json( body: { msg: 'Workspace not found' } );

    const isMember = workspace.members.some(m => m.user.toString() === req.user.id);
    if (!isMember) return res.status( code: 403 ).json( body: { msg: 'Access denied' } );

    const channels : Query<...> & ObtainSchemaGeneric<module:mon...> = await Channel.find( filter: { workspace: req.params.workspaceId }).populate( path: 'members', select: 'name email' );
    res.json(channels);
  } catch (err) {
    console.error(err.message);
    res.status( code: 500 ).send( body: 'Server Error' );
  }
}

```

Lorsqu'un utilisateur ouvre un canal, le frontend envoie une requête GET à l'endpoint /api/channels/:workspaceId/channels/:channelId/messages en incluant son JWT. Le serveur, via le middleware auth, vérifie son identité puis confirme qu'il appartient bien au workspace. Si l'accès est validé, il charge tous les messages MongoDB pour ce channelId, les trie par ordre chronologique (createdAt) et les renvoie en JSON. En cas d'erreur (workspace ou canal introuvable, droits insuffisants), l'API renvoie le code HTTP approprié (404 ou 403). Côté client, les messages sont alors affichés et enrichis en temps réel par les WebSockets.

```

85 router.get( path: './workspaceId/channels/:channelId/messages', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> ) : Promise<...> => {
86   try {
87     const workspace : Query<...> & ObtainSchemaGeneric<module.mon...> = await Workspace.findById(req.params.workspaceId);
88     if (!workspace) return res.status( code: 404 ).json( body: { msg: 'Workspace not found' } );
89
90     const isMember = workspace.members.some(m => m.user.toString() === req.user.id);
91     if (!isMember) return res.status( code: 403 ).json( body: { msg: 'Access denied' } );
92
93     const channel : Query<...> & ObtainSchemaGeneric<module.mon...> = await Channel.findById(req.params.channelId);
94     if (!channel) return res.status( code: 404 ).json( body: { msg: 'Channel not found' } );
95
96     const messages = await Message.find( filter: { channel: req.params.channelId }).sort( arg: { createdAt: 1 } );
97     res.json(messages);
98   } catch (err) {
99     console.error(err.message);
100     res.status( code: 500 ).send( body: 'Server Error' );
101   }

```

Notification en temps réel

Les notifications sont envoyées en temps réel dès lors que l'utilisateur est mentionné ou qu'il a reçu venant de Workspace. Cette fonction fonctionne grâce à une table correspondant aux utilisateurs qui enverra alors une notification si l'utilisateur est connecté.

```

12 router.get( path: '/', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> ) : Promise<void> => {
13   try {
14     const notifications = await Notification
15       .find({ user: req.user.id })
16       .sort({ createdAt: -1 })
17       .populate('fromUser', 'username');
18     res.json(notifications);
19   } catch (err) {
20     console.error(err.message);
21     res.status( code: 500 ).send( body: 'Server Error' );
22   }
23 });
24
25 //
26 * GET /api/notifications/:id
27 * Récupère une notification unique (pour pouvoir la peupler quand on la reçoit par socket).
28 */
29 router.get( path: './:id', auth, async (req : Request<P, ResBody, ReqBody, ReqQuery, LocalsObj> , res : Response<any, Record<...>> ) : Promise<...> => {
30   try {
31     const notif = await Notification
32       .findById(req.params.id)
33       .populate('fromUser', 'username');
34     if (!notif) return res.status( code: 404 ).json( body: { msg: 'Notification not found' } );
35     res.json(notif);
36   } catch (err) {
37     console.error(err.message);
38     res.status( code: 500 ).send( body: 'Server Error' );

```

Lorsque l'utilisateur sera connecté il verra une cloche en haut à droite avec le nombre de notifications non lue s'il a été mentionné. L'utilisateur clic sur la mention alors cela le renverra directement sur le bon canal et sur le message ou il a été mentionné avec une surbrillance bleue.

Api externe

Dans ce projet 4 API externes ont été utilisé dont 2 qui sont pour l'authentications, les 2 autres API sont GIPHY qui est une bibliothèque de GIPHS (Image animé), proposant une version gratuite de leurs API. Elle fonctionne grâce à Axios requêtes GET vers les giph concerné pour récupérer directement les liens des GIPH pour être intégré dans la discussion comme image.

```
// src/services/axiosGiphy.js
import axios from 'axios';

const axiosGiphy : AxiosInstance = axios.create({
  baseURL: 'https://api.giphy.com',
});

// Supprimer le header x-auth-token (hérité éventuellement d')
delete axiosGiphy.defaults.headers.common['x-auth-token'];

export default axiosGiphy; Show usages  👤 hugod
```

Concernant les émojis, nous avons choisi la bibliothèque open-source emoji mart, qui fournit de nombreux emoji disponible. Lorsqu'un utilisateur clique sur le bouton « Réagir » ou l'icône emoji, le picker s'ouvre et affiche l'ensemble des emoji classés par catégories. Au clic sur un emoji, on récupère son shortcode ou son caractère Unicode, qu'on injecte dans le contenu du message et qu'on envoie ensuite via l'API REST. L'usage d'Unicode natif garantit la compatibilité sur tous les devices (PC et mobile), tandis que la librairie gère automatiquement le rendu et la recherche rapide.

Barre de recherche avancé

La barre de recherche avancée repose sur un index full-textuel et création de n-grams pour supporter la recherche floue et par préfixe. Elle accepte une syntaxe enrichie grâce à des opérateurs booléens AND, OR, NOT, pour les expressions exactes et expose des dynamiques comme un utilisateur un message, un canal ou un Workspace. Grâce à un mécanisme d'auto-complétions, elle propose en temps réel des suggestions basées sur les recherches populaires ou l'historique personnel pour les mentions des utilisateurs ou les channels. Côté technique, elle communique via une API REST, pagine les résultats et met en cache les requêtes fréquentes pour optimiser la latence. On l'utilise typiquement pour retrouver rapidement des messages, des fichiers joints, des utilisateurs ou des canaux au sein de la plateforme, tout en permettant une recherche fine et rapide des données de l'utilisateur.

Recherche pour les mentions avec la commande « # » ou « / » :

```
router.get('/path', auth, async (req: Request<P, ResBody, ReqBody, ReqQuery, LocalsObj>, res: Response<any, Record<...>>) : Promise<...> => {  
  try {  
    const query = req.query.query?.trim() || '';  
    if (!query) {  
      return res.json({ body: {  
        channels: [],  
        users: [],  
        channelMessages: [],  
        dmMessages: [],  
      }});  
    }  
  }  
});
```

Recherche générales :

```
const channelMessages = await Message.find( filter: {
  content: { $regex: query, $options: 'i' },
})
.limit( val: 10)
.populate( path: 'sender', select: 'name email')
.populate( path: 'channel', select: 'name workspace') // pour récupérer le channelName
.sort( arg: { createdAt: -1 }); // tri par date desc ?

// 4) Rechercher dans les DirectMessages
const dmMessages = await DirectMessage.find( filter: { content: { $regex: query, $options: 'i' } })
.limit( val: 10)
.populate( path: 'sender', select: 'name email')
.populate( path: 'receiver', select: 'name email')
.sort( arg: { createdAt: -1 });

// Renvoyer un objet
return res.json( body: {
  channels,
  users,
  channelMessages,
  dmMessages,
});
```

Interface utilisateur

L'application utilise le framework React, nous avons décidé de l'utiliser pour son efficacité à gérer les interfaces utilisateurs dynamiques. Ce qui assure une expérience de l'utilisateur très fluide avec une gestion optimisée en facilitant les interactions au sein de l'application.

Avantages :

- Composant réutilisables : pour avoir une meilleure cohérence visuelle et fonctionnelle dans toute l'application.
- Routing avancé : Une meilleure gestion de la navigation au sein des Workspace, canaux et messages privés.
- Thèmes dynamiques : Les utilisateurs peuvent changer l'interface utilisateur avec plusieurs thèmes à l'heure disposition (dark, light, camel & purple) qui sauvegarde automatiquement dans le profil de l'utilisateur dans la bdd.
- Optimisation de performance : Techniques comme le lazy-loading, mémorisation des composants.

Développement mobile

L'application mobile SUPCHAT, a été créée en React Native, reposant sur un Stack Navigator, qui définit le composant AppNavigator. Il est enveloppé par un NavigationContainer, qui gère l'état global de la navigation et l'historique des écrans. Les premières routes du stack correspondent au parcours d'authentification et d'initialisation, qui va commencer par l'écran InitScreen, puis l'utilisateur peut accéder à LoginScreen, RegisterScreen, ForgotPasswordScreen ou ResetPasswordScreen selon les cas. Chaque écran reçoit, le cas échéant, des paramètres via la définition TypeScript du RootStackParamList. Pour la suite des fonctionnalités il reprendra les API créées sur la version Web en adaptant le front avec du typescript.